

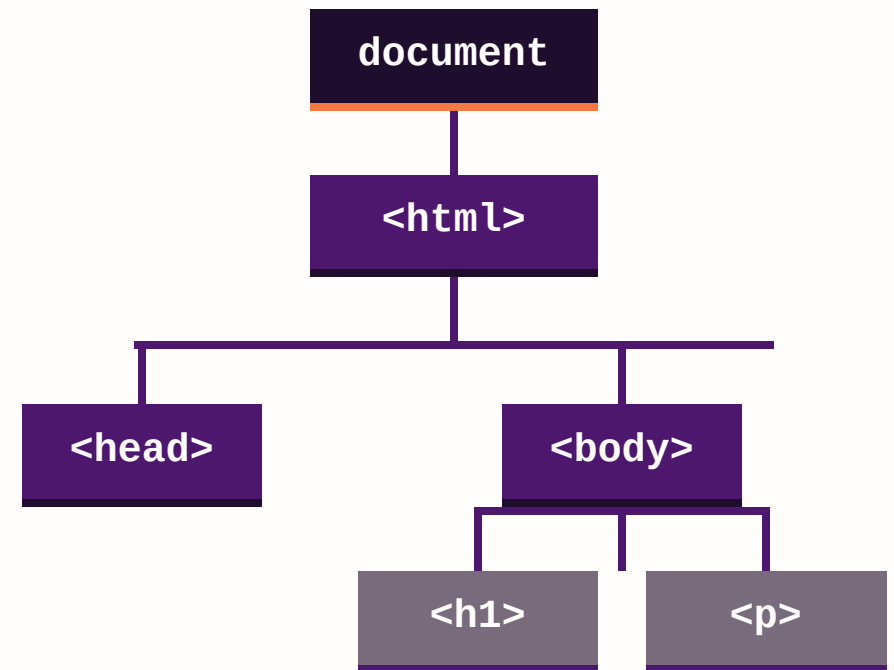
MODULE 03

JavaScript DOM: Connecting Code to the Page

How the browser turns HTML into a tree of JavaScript objects, and how we use code to manipulate it.

LEA LABS · WEB DEVELOPMENT

THE DOM TREE



Selecting Elements

Before you can change an element, you have to find it. We use the exact same CSS selectors you already know to grab elements from the DOM.

QUERYSELECTOR()

Finds the **first** element that matches the CSS selector. If no element is found, it returns `null`. Perfect for unique elements like headers or specific IDs.

QUERYSELECTORALL()

Finds **all** elements that match the selector and returns them in a `NodeList` (which acts a lot like an array). Perfect for lists, cards, or repeating buttons.

JAVASCRIPT SELECTION

```
// Select by ID (returns one element)
const title = document.querySelector('#main-
title');
// Select by Class (first match)
const firstBtn =
document.querySelector('.btn');
// Select ALL by Class (list)
const allBtns =
document.querySelectorAll('.btn');
```

Changing Content and Attributes

Select an element to read or update its text, HTML, or attributes via the DOM.

TEXTCONTENT VS INNERHTML

Use `textContent` for safe plain-text. Use `innerHTML` when you need to insert HTML structure.

STANDARD ATTRIBUTES

Common attributes like `src`, `href`, `id`, and `value` map to DOM properties.

UPDATING TEXT AND HTML

```
const heading =  
document.querySelector('h1');  
heading.textContent = "Welcome to LEA  
Labs";
```

UPDATING ATTRIBUTES

```
const image =  
document.querySelector('img');  
image.src = "new-logo.png";  
const link = document.querySelector('a');  
link.href = "https://lealabs.com";
```

Changing Styles and Classes

Change element appearance with inline styles or—preferably—by toggling CSS classes.

THE STYLE PROPERTY

Apply inline CSS from JavaScript. Hyphenated properties like `background-color` become `backgroundColor`.

THE CLASSLIST API

Use `add()`, `remove()`, and `toggle()` to manage classes without overwriting other styles.

INLINE STYLES

```
const box =
document.querySelector('.box');

// Sets style="background-
// color: purple;"
box.style.backgroundColor =
"purple";
```

MANAGING CLASSES

```
const menu =
document.querySelector('.menu');

// Add or toggle a class
menu.classList.add('active');
menu.classList.toggle('expanded');
```

Creating New Elements

You can build entirely new HTML elements from scratch using JavaScript. It takes three steps: create the element, configure it, and attach it to the page.

1 CREATE

Use `document.createElement()` to make a new, empty HTML tag in memory.

2 CONFIGURE

Add text, classes, and attributes to your new element before it appears on screen.

3 ATTACH

Use `appendChild()` or `append()` to insert the element into an existing parent in the DOM.

THE 3-STEP PROCESS

```
// 1. Create const newCard =  
document.createElement('div');  
  
// 2. Configure  
newCard.classList.add('course-card');  
newCard.innerHTML = '<h3>JavaScript  
DOM</h3><p>Learn to build interfaces.</p>';  
  
// 3. Attach const container =  
document.querySelector('.grid');  
container.appendChild(newCard);
```

Removing Elements

When an element is no longer needed—like a dismissed alert or a deleted to-do item—you can safely remove it from the DOM tree.

ELEMENT.REMOVE()

The modern, direct way to remove an element. Call this on the element you want to delete.

PARENT.REMOVECHILD()

The traditional method: select the parent and tell it to remove a specific child.

THE MODERN WAY

```
const alertBox =  
document.querySelector('.alert');  
  
// element removes itself  
alertBox.remove();
```

THE TRADITIONAL WAY

```
const list =  
document.querySelector('ul');  
const firstItem =  
list.querySelector('li');  
  
// parent removes the child  
list.removeChild(firstItem);
```

Listening for Events

The DOM allows us to listen for user interactions (clicks, typing, scrolling) and run JavaScript when they happen.

ADDEVENTLISTENER()

The modern way to attach events. It takes two arguments: the name of the event (like 'click') and a callback function to run when the event occurs.

SEPARATION OF CONCERNS

By attaching events in JavaScript, we keep our HTML clean. We avoid using old inline attributes like `onclick=""` directly in our markup.

ATTACHING A CLICK EVENT

```
const button =  
document.querySelector('.btn');  
  
// select and attach listener  
button.addEventListener('click',  
function() {  
  console.log('Button was clicked!');  
  button.classList.add('active');  
});
```

The Event Object

When an event occurs, the browser automatically passes an "event object" to your callback function containing details about what just happened.

THE "E" PARAMETER

By adding a parameter (commonly `e`) to your callback, you receive an object with data like mouse coords or which key was pressed.

E.TARGET

The most useful property. `e.target` points to the HTML element that triggered the event — handy for reading values or modifying the clicked element.

CATCHING THE EVENT

```
document.addEventListener('keydown',  
function(e) {  
  console.log('Key pressed:', e.key);  
});
```

USING E.TARGET

```
const input =  
document.querySelector('input');  
input.addEventListener('input', function(e)  
{  
  const currentText = e.target.value;  
  console.log(currentText);  
});
```

Handling Form Submissions

Forms refresh on submit by default. Use JS to stop the reload and handle the data client-side.

E.PREVENTDEFAULT()

Prevents the browser's default action so our code can process the form without a page reload.

THE FORMDATA API

Use new `FormData(form)` to collect all input values at once without selecting each field.

INTERCEPTING A SUBMIT EVENT

```
const form =
document.querySelector('#login');
form.addEventListener('submit', function(e)
{
  e.preventDefault();
  const data = new FormData(form);
  const email = data.get('email');
  console.log(`Logging in: ${email}`);
});
```

Event Delegation

Instead of attaching an event listener to every single child element, attach one listener to their parent and let events "bubble" up.

1. BETTER PERFORMANCE

Attaching 100 listeners to 100 list items uses more memory.
Attaching 1 listener to the parent list is highly efficient.

2. DYNAMIC ELEMENTS

If you add new items to the list later via JavaScript, they automatically inherit the behavior because the parent is still listening.

DELEGATING TO THE PARENT

```
const list = document.querySelector('ul');
list.addEventListener('click', function(e)
{
  if (e.target.matches('li')) {

    e.target.classList.toggle('completed');
  }
});
```

Lab: Interactive To-Do List

Turn static HTML into a working application. Use the DOM API and event listeners to build a task tracker.

DELIVERABLES

- Capture form submissions to add tasks.
- Create DOM elements for each task and append them.

SUCCESS CRITERIA

- Prevent page reloads (e.preventDefault()).
- Use event delegation for task actions.
- Update the DOM without duplicating items.

EXTENSION CHALLENGE

Add simple filter buttons: All, Active, Completed.